11. Структура Go проекта

Введение в организацию Go проектов

Зачем нужна структура проекта?

- **Масштабируемость** — по мере роста проекта нужна четкая организация
- **Читаемость** — другие разработчики должны легко понимать код

  Переиспользование — разделение на модули и пакеты
- **Сопровождение** — упрощение поддержки и развития проекта

Стандартный макет Go проекта

Важно понимать:

- Это **НЕ официальный стандарт** от команды Go
- Это **исторически сложившиеся практики** сообщества
- Используется в большинстве крупных Go проектов
- Не обязательно применять все директории сразу

Принципы организации:

1. **Разделение ответственности** — каждая директория имеет свою цель
2. **Явные намерения** — структура показывает назначение кода
3. **Переиспользование** — четкое разделение на внутренний и публичный код

Основные директории Go

/cmd — Точки входа в приложения

```
/cmd/
  /myapp/ main.go

  /worker/ main.go


  /cli/

    main.go
```

Правила:

- Имя директории = имя исполняемого файла
- Минимум кода в `main.go`
- Импорт логики из `/internal` и `/pkg`

Пример структуры main.go:

```
package main


import (
    "github.com/myproject/internal/app"
    "github.com/myproject/internal/config"
)


func main() {
    cfg := config.Load() app.Run(cfg)
}
```

/internal — Внутренний код

Назначение: Код, который НЕ должен использоваться в других проектах.

Компилятор Go **запрещает** импорт из `/internal` в других проектах

```
/internal/        # Основное место для бизнес-
                  логики
  /app/           # Логика приложения

    /myapp/

  /pkg/           # Внутренние библиотеки

    /database/

    /auth/

  /config/        # Конфигурация

  /handlers/      # HTTP обработчики

  /services/      # Бизнес-логика

  /repository/    # Слой данных
```

/pkg — Публичные библиотеки

Назначение: Код, который могут использовать другие проекты

```
/pkg/
   /logger/            # Логирование
   /validator/         # Валидация
   /utils/             # Утилиты
   /client/            # API клиенты
```

Когда использовать:

✅ Утилиты общего назначения
✅ SDK и клиенты
❌ Бизнес-логика конкретного приложения

Директории для зависимостей

/vendor

```
# Создание vendor директории
go mod vendor
```

Когда использовать:

- ✅ Контроль над зависимостями
- ✅ Offline сборки
- ❌ Библиотеки (не коммитить vendor)

Директории для сервисов и API

/api — API спецификации

Содержимое:

- OpenAPI/Swagger файлы
- Protocol Buffers (`.proto`)
- GraphQL схемы

```
/api/
  /openapi/

    api.yaml

  /protobuf/

    user.proto


  /graphql/

    schema.graphql
```

/web — Веб-ресурсы

Для веб-приложений:

```
/web/
  /static/        # CSS, JS, изображения
    /css/
    /js/
    /images/
  /templates/     # HTML шаблоны
  /spa/           # Single Page Application
```

Конфигурация и развертывание

/configs — Файлы конфигурации

```
/configs/
  config.yaml

  config.prod.yaml

  config.dev.yaml

  docker-compose.yml
```

/deployments — Развертывание

```
/deployments/
  /kubernetes/

    deployment.yaml service.yaml

  /docker/

   Dockerfile

   docker-compose.yml

  /terraform/

     main.tf
```

/scripts — Скрипты автоматизации

```
/scripts/
  build.sh

  deploy.sh

  migrate.sh

  test.sh
```

Преимущества:

- Простой Makefile
- Переиспользуемые скрипты
- Автоматизация CI/CD

Тестирование и документация

/test — Тестовые данные

```
/test/
  /data/            # Тестовые данные
  /fixtures/          # Фикстуры
  /integration/ # Интеграционные тесты
  /e2e/     # End-to-end тесты
```

Соглашения Go:

- Файлы с `_test.go` — unit тесты рядом с кодом
- `/test` — для больших тестовых наборов
- `testdata/` — игнорируется Go

/docs — Документация

```
/docs/
  README.md CONTRIBUTING.md


  /api/   # API документация

  /architecture/ # Архитектурные решения

  /deployment/  # Инструкции по развертыванию
```

Вспомогательные директории

/tools — Инструменты разработки

```
/tools/
  /codegen/      # Генераторы кода
  /migration/    # Миграции БД
  /linter/       # Кастомные линтеры
```

Особенности:

- Могут импортировать из `/pkg` и `/internal`
- Обычно не входят в основную сборку
- Помогают в разработке

/examples — Примеры использования

```
/examples/
  /basic/ main.go

  /advanced/ main.go

  /integration/ main.go
```

/third_party — Сторонние инструменты

```
/third_party/
  /swagger-ui/  # Swagger UI
  /grafana/         # Дашборды Grafana
```

Пример реального Go проекта

```
myproject/
├── cmd/                    # Основные приложения
│   ├── server/main.go
│   ├── worker/main.go
│   └── cli/main.go
├── internal/               # Внутренний код
│   ├── app/
│   │   ├── server/
│   │   └── worker/
│   ├── config/
│   ├── handlers/
│   ├── services/
│   ├── repository/
│   └── models/
├── pkg/                    # Публичные библиотеки
│   ├── logger/
│   ├── validator/
│   └── client/
├── api/                    # API спецификации
│   ├── openapi/
│   └── protobuf/
├── web/                    # Веб-ресурсы #
├── configs/                Конфигурация
├── deployments/            # Развертывание
├── scripts/                # Скрипты
├── tests/                  # Интеграционные или апи тесты
├── docs/                   # Документация
├── tools/                  # Инструменты
├── .gitignore
├── go.mod (go.sum)
├── Makefile
└── README.md
```

Практические примеры

Простое CLI приложение

```
mycli/
├── cmd/
│   └── mycli/
│       └── main.go
├── internal/
│   ├── commands/
│   ├── config/
│   └── utils/
├── go.mod
├── go.sum
└── README.md
```

Микросервис

```
userservice/
├── cmd/
│   └── server/
│       └── main.go
├── internal/
│   ├── handlers/
│   ├── services/
│   ├── repository/
│   └── models/
├── api/
│   └── openapi/
│       └── users.yaml
├── deployments/
│   └── kubernetes/
├── configs/
├── go.mod
└── go.sum
```

Библиотека

```
mylib/              # Или прямо в корне
├── pkg/
│   ├── auth/
│   ├── crypto/
│   └── utils/
├── examples/
├── docs/
├── go.mod
├── go.sum
└── README.md
```

Антипаттерны и ошибки

Директория `/src`

Не используйте:

```
myproject/
├── src/            # ❌ Java-паттерн
│   └── main.go
```

Почему плохо:

- Это Java-паттерн, не Go
- Путаница с `$GOPATH/src`
- Лишний уровень вложенности

Все в одной директории

```
myproject/
├── main.go        # ❌ Все в корне
├── user.go
├── order.go
├── database.go
└── handler.go
```

Неправильное использование `/pkg`

```
/pkg/
├── handlers/      # ❌ Специфично для приложения
├── models/        # ❌ Внутренние модели
└── database/      # ❌ Конкретная БД
```

Лучшие практики

1. **Начинайте просто**
   - Один файл `main.go` для обучения
   - Добавляйте структуру по мере роста
   - Не переусложняйте маленькие проекты

2. **Четкое разделение ответственности**

   - `/cmd` — только точки входа
   - `/internal` — вся бизнес-логика
   - `/pkg` — только переиспользуемое

3. Именование и организация

- Понятные имена директорий
- Логическая группировка файлов
- Консистентность в проект

4. Документирование

- README с описанием структуры
- Комментарии к нестандартным решениям
- Примеры использования

5. Инструментарий

- Используйте `gofmt` , `golint`
- Настройте CI/CD
- Автоматизируйте рутинные задачи

Примеры из реального мира

Крупные проекты с хорошей структурой:

1. **Kubernetes** — https://github.com/kubernetes/kubernetes
2. **Docker** — https://github.com/docker/docker
3. **Terraform** — https://github.com/hashicorp/terraform
4. **Prometheus** — https://github.com/prometheus/prometheus
5. **Grafana** — https://github.com/grafana/grafana